ISO/IEC JTC1 SC22 WG21 P0067R1
Group: LEWG
Jens Maurer <Jens.Maurer@gmx.net>
2016-02-12

P0067R1: Elementary string conversions, revision 1

Introduction

Following up on N4412 "Shortcomings of iostreams", this paper presents low-level, locale-independent functions for conversions between integers and strings and between floating-point numbers and strings.

Use cases include the increasing number of text-based interchange formats such as JSON or XML that do not require internationalization support, but do require high throughput when produced by a server.

There are a lot of existing functions in C++ to perform such conversions, but none offers a high-performance solution. At a minimum, an implementation by an ordinary user of the language using an elementary textbook algorithm should not be able to outperform a quality standard library implementation. The requirements are thus:

- no runtime parsing of format strings
- no dynamic memory allcoation inherently required by the interface
- no consideration of locales
- no indirection through function pointers required
- prevention of buffer overruns
- when parsing a string, errors are distinguishable from valid numbers
- when parsing a string, whitespace or decorations are not silently ignored

For floating-point numbers, there should be a facility to output a floating-point number with a minimum number of decimal digits where input from the digits is guaranteed to reproduce the original floating-point value.

The deliberations in the Kona LEWG sessions resulted in the following comments:

- cover all the formatting facilities of printf (choice of base, scientific, precision,hexfloat)
- spell out round-trip guarantees for the decimal format
- explain cost of runtime parameter for base vs. having a separate overload with fixed base = 10
- use the names to_chars and from_chars
- possibly use interface struct { char* ptr; bool overflow; } to_chars(char* ptr, char* end, T value);
- use interface struct { const char* ptr; error_code ec; } from_chars(const char* ptr, const char* end, T& value);
- check use cases to nail down the to_chars interface

Changes

Compared to P0067R0

- Addressed comments from LEWG deliberations in Kona (blue print)
- Added open issues

Existing approaches

C++ already provides at least the facilities in the following table, each with shortcomings highlighted in the second column.

facility	shortcomings
sprintf	format string, locale, buffer overrun
snprintf	format string, locale
sscanf	format string, locale
atol	locale, does not signal errors
strtol	locale, ignores whitespace and 0x prefix
strstream	locale, ignores whitespace

stringstream	locale, ignores whitespace, memory allocation
num_put / num_get facets	locale, virtual function
to_string	locale, memory allocation
stoi etc.	locale, memory allocation, ignores whitespace and 0x prefix, exception on error

As a rough performance comparison, the following simple numeric formatting task was implemented: Output the integer numbers 0 ... 1 million, separated by a single space character, into a contiguous array buffer of 10 MB. This task was executed 10 times. The execution environment was gcc 4.9 on Intel Core i5 M450.

strstream	864 ms	uses `std::strstream` with application-provided buffer
streambuf	540 ms	uses simple custom streambuf with `std::num_put<>` facet
direct	285 ms	open-coded "divide by 10" algorithm, using the interface described below
fixed-point	125 ms	fixed-point algorithm found in an older AMD optimization guide, using the interface described below

There are various approaches for even more efficient algorithms; see, for example, https://gist.github.com/anonymous/7700052 .

Interface discussion

The following discussion assumes that a common interface style should be established that covers (built-in) integer and floating-point types. The type `T` designates such an arithmetic type. Note that given these restrictions, output of T to a string has a small maximum length in all cases. The styles for input vs. output will differ due to the differing functionality.

The fundamental interface for a string is that it is caller-allocated, contiguous in memory, and not necessarily 0-terminated. That means, it can be represented by a range [`begin`,`end`) where `begin` and `end` are of type `char *`.

Given this framework, the following subsections discuss various specific interface styles for both output and input. In each case, the signature of an integer output or input function is shown. Criteria for comparison include impact on compiler optimizations, indication of output buffer overflow, and composability (as a measure of ease-of-use).

Output

This subsection discusses various specific interface styles for output. In each case, the signature of an integer output function is shown. There is one failure mode for output: overflow of the provided output buffer. Criteria for comparison include impact on compiler optimizations, indication of output buffer overflow, and composability (as a measure of ease-of-use). For exposition of the latter, consecutive output of two numbers is shown, without any separator.

Conceptually, an output function has four parameters and two results. The parameters are the `begin` and `end` pointers of the buffer, the value, and the desired base. The results are the updated `begin` pointer and an overflow indication.

Feature set of printf / strtol for integers

base 2...36	overload provided
uppercase for base > 10	not supported

Feature set of printf / strtod for floating-point

The following table lists the format specifiers of fprintf relevant to floating-point in C11 and the disposition in the context of the functionality proposed in this paper.

	field width	not supported
	precision (number of digits after the decimal-point)	overload provided
+	mandatory sign	not supported
space	prefix	not supported
#	mandatory decimal point	not supported
0	pad with zeroes	not supported
L	long double argument	overload provided

f	fixed-precision lowercase conversion	overload provided
F	fixed-precision uppercase conversion	not provided
e	scientific lowercase conversion	overload provided
E	scientific uppercase conversion	not provided
g	switch between f and e	not provided
G	switch between F and E	not provided
a	hexadecimal lowercase conversion	overload provided
A	hexadecimal uppercase conversion	not provided

Iterator

```
char * to_chars(char * begin, char * end, T value, int base = 10);
```

This interface style returns the updated `begin` pointer. That is, the resulting string is in [`begin`, *return-value*) and [*return-value*, `end`) is unused space in the string. Such an interface style is used for many standard library algorithms, e.g. `find` [alg.find]. All parameters are passed by value which helps the optimizer. Overflow is indicated by *return-value* == `end`. The situation that the output exactly fits into the provided buffer cannot be distinguished from overflow. Two consecutive outputs can be produced trivially using:

```
p = to_chars(p, end, value1);
p = to_chars(p, end, value2);
```

Iterator with in-situ update

```
void to_chars(char *& begin, char * end, T value, int base = 10);
```

This interface style updates the `begin` pointer in place. That is, the resulting string is in [`old-begin`, `begin`) and [`begin`,`end`) is unused space in the string. Aliasing rules allow that updates to `begin` change the data where begin points. To avoid redundant updates, the implementation can copy `begin` to a local variable. Overflow is indicated by `begin` reaching `end`. The situation that the output exactly fits into the provided buffer cannot be distinguished from overflow. Two consecutive outputs can be produced trivially using:

```
to_chars(p, end, value1);
to_chars(p, end, value2);
```

string_view

```
void to_chars(std::string_view& s, T value, int base = 10);
```

This interface style groups the `begin` and `end` pointers into a `string_view` which is updated in-place. Comments on "iterator with in-situ update" apply analogously.

Iterator with in-situ update and overflow indication

Adding a boolean return value allows to indicate overflow:

```
bool to_chars(char *& begin, char * end, T value, int base = 10);
```

Comments on "iterator with in-situ update" apply analogously, except that the return value indicates whether overflow occurred.

snprintf

```
int to_chars(char * begin, char * end, T value, int base = 10);
```

This interface style always returns the number of characters required to output T, regardless of whether sufficient space was provided. That is, an overflow occurred if the return value is larger than `end-begin`, otherwise the resulting string is in [`begin`, `begin + return-value`). Such an interface style is used for `snprintf`, except that the proposed function never 0-terminates the output. All parameters are passed by value which helps the optimizer. Overflow is indicated by a return value strictly larger than the distance between `begin` and `end`. Computing the amount of overflow is helpful to allocate a larger buffer, but, in general, requires switching from the fast path, because no further characters may be stored. The elementary functions discussed in this paper all have (statically computable) limited maximum output size, so the benefit of returning the exact size is small. Two consecutive outputs require attention at the caller site to avoid buffer overflow:

```
int n = 0;
n += to_chars(begin, end, value1);
n += to_chars(begin + std::min(n, end-begin), end, value2);
```

Kona consensus

```
struct to_chars_result {
  char* ptr;
  bool overflow;
  operator tuple<char *, bool>() const;
};
char* get<0>(const to_chars_result&);   // for tie()
bool get<1>(const to_chars_result&);
to_chars_result to_chars(char* begin, char* end, T value, int base = 10);
```

This interface style returns a named pair with the updated `begin` pointer. All parameters are passed by value which helps the optimizer. Overflow is indicated by a separate overflow indicator in the return value. Two consecutive outputs can be produced easily using:

```
to_chars_result result = to_chars(p, end, value1);
result = to_chars(result.ptr, end, value2);
```

Input

An input function conceptually operates in two steps: First, it consumes characters from the input string matching a pattern until the first non-matching character or the end of the string is encountered. Second, the matched characters are translated into a value of type `T`. There are two failure modes: no characters match, or the pattern translates to a value that is not in the range representable by `T`.

Conceptually, an input function has three parameters and three results. The parameters are the `begin` and `end` pointers of the string and the desired base. The results are the updated `begin` pointer, a `std::error_code` and the parsed value.

This subsection discusses various specific interface styles for input. Failure is indicated by `std::error_code` with the appropriate value. In each case, the signature of an integer input function is shown. Criteria for comparison include impact on compiler optimizations and composability (as a measure of ease-of-use). For exposition of the latter, parsing of two consecutive values is shown, without skipping of any separator.

Iterator

```
const char * from_chars(const char * begin, const char * end, T& value, std::error_code& ec, int base = 10);
```

This interface style returns the updated `begin` pointer. That is, the returned pointer points to the first character not matching the pattern. Such an interface style is used for many standard library algorithms. Two consecutive inputs can be performed like this:

```
T value1, value2;
std::error_code ec;
p = from_chars(p, end, value1, ec);
if (ec)
  /* parse error */;
p = from_chars(p, end, value2, ec);
if (ec)
  /* parse error */;
```

Iterator with in-situ update

```
void from_chars(const char *& begin, const char * end, T& value, std::error_code& ec, int base = 10);
```

This interface style updates the `begin` pointer in place. Two consecutive inputs can be performed like this:

```
T value1, value2;
std::error_code ec;
from_chars(p, end, value1, ec);
if (ec)
  /* parse error */;
from_chars(p, end, value2, ec);
if (ec)
  /* parse error */;
```

Iterator with in-situ update and error return

```
std::error_code from_chars(const char *& begin, const char * end, T& value, int base = 10);
```

Returning the error code allows for more compact code at the call site:

```
T value1, value2;
if (std::error_code ec = from_chars(p, end, value1))
  /* parse error */;
if (std::error_code ec = from_chars(p, end, value2))
  /* parse error */;
```

Return a std::pair or std::tuple

Two of the three results of an input function could be represented by a pair. All three results could be represented by a tuple. However, experience with `std::map` shows that the naming of the parts (`first` and `second`) carries no semantic meaning which would help reading the resulting code. If the result value moves to the return value, its type `T` needs to be passed explicitly (e.g. as a template parameter). The composition example would be:

```
std::pair<T, std::error_code> res;
res = from_chars<T>(p, end);
if (res.second)
  /* parse error */;
T value1 = res.first;
res = from_chars<T>(p, end);
if (res.second)
  /* parse error */;
T value2 = res.second;
```

Kona consensus

```
struct from_chars_result {
  const char* ptr;
  error_code ec;
};
const char * get<0>(const from_chars_result&); // for tie()
error_code get<1>(const from_chars_result&);
from_chars_result from_chars(const char* begin, const char* end, T& value, int base = 10);
```

This interface style returns the updated `begin` pointer and an error code. All parameters, except for the parsed value, are passed by value, which helps the optimizer. Two consecutive inputs can be performed like this:

```
T value1, value2;
from_chars_result result = from_chars(p, end, value1);
if (result.ec)
  /* parse error */
result = from_chars(result.ptr, end, value2);
if (result.ec)
  /* parse error */
```

Naming

The LEWG deliberations in Kona expressed the following naming preferences (sorted by number of votes).

to_text	9
to_chars	9
to_digits	7
to_characters	7
to_printable	6
to_ascii	3
to_string	3
to_output	1
[de]serialize	1
[un]marshal	1
[de]stringify	1

Given the tie in the first place and the author's personal preference for `to_chars`, this paper proposes `to_chars` for the output function and `from_chars` for the input (parse) function.

Cost of runtime parameter for base vs. separate overloads

The alternatives are

```
to_chars_result to_chars(char* begin, char* end, T value, int base = 10);
```

vs.

```
to_chars_result to_chars(char* begin, char* end, T value);
to_chars_result to_chars(char* begin, char* end, T value, int base);
```

The difference is almost a quality-of-implementation issue, except that the standard gives appropriate liberty only for member functions, not for non-member functions (17.6.5.5 [member.functions]). The former can be implemented like this:

```
inline to_chars_result to_chars(char* begin, char* end, T value, int base = 10)
{
  if (base == 10)
    return to_chars2(begin, end, value);
  else
    return to_chars2(begin, end, value, base);
}
```

Other than a slightly increased burden on the inlining and constant propagation capabilities of the compiler, the two signatures are thus identical in performance. I have analyzed similar cases in the past and can confirm that the inline function essentially vanishes for optimized compiles. Personally, I would prefer to give an implementation latitude to switch between the two interface styles as it sees fit, but that is a question that should be discussed in a wider context, independent of the present paper. A similar argument applies to the question of overhead for base = 16, where a very efficient implementation using SIMD vector instructions is possible.

Minor concerns

- Why do we not throw an exception on parse error? Two reasons: Exceptions come with a cost (in particular, when thrown), and a parse error is not an exceptional situation.

Wording

Header synopsis

```
namespace std {
  struct to_chars_result {
    char* ptr;
    bool overflow;
  };
  char * get<0>(const to_chars_result&);
  bool get<1>(const to_chars_result&);

  to_chars_result to_chars(char* begin, char* end,   signed char      value, int base = 10);
  to_chars_result to_chars(char* begin, char* end, unsigned char      value, int base = 10);
  to_chars_result to_chars(char* begin, char* end,   signed short     value, int base = 10);
  to_chars_result to_chars(char* begin, char* end, unsigned short     value, int base = 10);
  to_chars_result to_chars(char* begin, char* end,   signed int       value, int base = 10);
  to_chars_result to_chars(char* begin, char* end, unsigned int       value, int base = 10);
  to_chars_result to_chars(char* begin, char* end,   signed long      value, int base = 10);
  to_chars_result to_chars(char* begin, char* end, unsigned long      value, int base = 10);
  to_chars_result to_chars(char* begin, char* end,   signed long long value, int base = 10);
  to_chars_result to_chars(char* begin, char* end, unsigned long long value, int base = 10);

  to_chars_result to_chars(char* begin, char* end, float       value, bool hex = false);
  to_chars_result to_chars(char* begin, char* end, double      value, bool hex = false);
  to_chars_result to_chars(char* begin, char* end, long double value, bool hex = false);

  enum class to_chars_format {
    fixed, scientific, hex
  };

  to_chars_result to_chars(char* begin, char* end, float       value, to_chars_format fmt, int precision = 6);
  to_chars_result to_chars(char* begin, char* end, double      value, to_chars_format fmt, int precision = 6);
  to_chars_result to_chars(char* begin, char* end, long double value, to_chars_format fmt, int precision = 6);


  struct from_chars_result {
    const char* ptr;
    error_code ec;
  };
  const char * get<0>(const from_chars_result&);
  error_code get<1>(const from_chars_result&);
```

```
  from_chars_result from_chars(const char* begin, const char* end, signed char& value, int base = 10);
  from_chars_result from_chars(const char* begin, const char* end, unsigned char& value, int base = 10);
  from_chars_result from_chars(const char* begin, const char* end, signed short& value, int base = 10);
  from_chars_result from_chars(const char* begin, const char* end, unsigned short& value, int base = 10);
  from_chars_result from_chars(const char* begin, const char* end, signed int& value, int base = 10);
  from_chars_result from_chars(const char* begin, const char* end, unsigned int& value, int base = 10);
  from_chars_result from_chars(const char* begin, const char* end, signed long& value, int base = 10);
  from_chars_result from_chars(const char* begin, const char* end, unsigned long& value, int base = 10);
  from_chars_result from_chars(const char* begin, const char* end, signed long long& value, int base = 10);
  from_chars_result from_chars(const char* begin, const char* end, unsigned long long& value, int base = 10);

  from_chars_result from_chars(const char* begin, const char* end, float& value, bool hex = false);
  from_chars_result from_chars(const char* begin, const char* end, double& value, bool hex = false);
  from_chars_result from_chars(const char* begin, const char* end, long double& value, bool hex = false);
}
```

Output functions

All functions named `to_chars` convert `value` into a character string by successively filling the range [`begin`, `end`). If the member `overflow` of the return value is `false`, the conversion was successful and the member `ptr` is the one-past-the-end pointer of the characters written. Otherwise, the member `ptr` has the value `end` and the contents of the range [`begin`, `end`) is unspecified.

```
  to_chars_result to_chars(char* begin, char* end,   signed char      value, int base = 10);
  to_chars_result to_chars(char* begin, char* end, unsigned char      value, int base = 10);
  to_chars_result to_chars(char* begin, char* end,   signed short     value, int base = 10);
  to_chars_result to_chars(char* begin, char* end, unsigned short     value, int base = 10);
  to_chars_result to_chars(char* begin, char* end,   signed int       value, int base = 10);
  to_chars_result to_chars(char* begin, char* end, unsigned int       value, int base = 10);
  to_chars_result to_chars(char* begin, char* end,   signed long      value, int base = 10);
  to_chars_result to_chars(char* begin, char* end, unsigned long      value, int base = 10);
  to_chars_result to_chars(char* begin, char* end,   signed long long value, int base = 10);
  to_chars_result to_chars(char* begin, char* end, unsigned long long value, int base = 10);
```

Requires: `base` has a value between 2 and 36 (inclusive).

Effects: The value of `value` is converted to a string of digits in the given base (with no redundant leading zeroes). Digits in the range 10..36 (inclusive) are represented as lowercase characters a..z. If `value` is less than zero, the representation starts with a minus sign.

```
  to_chars_result to_chars(char* begin, char* end, float       value, bool hex = false);
  to_chars_result to_chars(char* begin, char* end, double      value, bool hex = false);
  to_chars_result to_chars(char* begin, char* end, long double value, bool hex = false);
```

Effects: If `hex` is false, a finite value is converted to string matching a *floating-literal* without *floating-suffix* and with lowercase `e` leading the *exponent-part* (if present); see 2.13.4 [lex.fcon]. Otherwise, a finite value is converted to a string as-if by `printf("%a")` in the "C" locale (without leading "0x"). In either case, the representation is such that there is at least one digit before the radix point (if present) and it requires a minimal number of characters, yet parsing the representation using the corresponding `from_chars` function recovers `value` exactly [Footnote: This guarantee applies only if to_chars and from_chars is executed on the same implementation.]. If value is not a finite value, `value` is converted to an implementation-defined string.

```
  to_chars_result to_chars(char* begin, char* end, float       value, to_chars_format fmt, int precision = 6);
  to_chars_result to_chars(char* begin, char* end, double      value, to_chars_format fmt, int precision = 6);
  to_chars_result to_chars(char* begin, char* end, long double value, to_chars_format fmt, int precision = 6);
```

Effects: `value` is converted to a string as-if by `printf` in the "C" locale with the given precision and using the format specifier `%f` if `fmt` is `to_chars_format::fixed`, `%e` if `fmt` is `to_chars_format::scientific`, and `%a` (without leading "0x") if `fmt` is `to_chars_format::hex`.

Input functions

All functions named `from_chars` analyze the string [begin,end) for a pattern. If no characters match the pattern, `value` is unmodified, the member `ptr` of the return value is `begin` and the member `ec` is EINVAL. Otherwise, the characters matching the pattern are interpreted as a representation of a value of type T. The member `ptr` of the return value points to the first character not matching the pattern, or has the value `end` if all characters match. If the parsed value is not in the range representable by the type of `value`, `value` is unmodified and the member `ec` is set to ERANGE. Otherwise, `value` is set to the parsed value and the member `ec` is set such that the conversion to `bool` yields false.

```
  from_chars_result from_chars(const char* begin, const char* end, signed char& value, int base = 10);
  from_chars_result from_chars(const char* begin, const char* end, unsigned char& value, int base = 10);
  from_chars_result from_chars(const char* begin, const char* end, signed short& value, int base = 10);
  from_chars_result from_chars(const char* begin, const char* end, unsigned short& value, int base = 10);
  from_chars_result from_chars(const char* begin, const char* end, signed int& value, int base = 10);
  from_chars_result from_chars(const char* begin, const char* end, unsigned int& value, int base = 10);
  from_chars_result from_chars(const char* begin, const char* end, signed long& value, int base = 10);
```

```
from_chars_result from_chars(const char* begin, const char* end, unsigned long& value, int base = 10);
from_chars_result from_chars(const char* begin, const char* end, signed long long& value, int base = 10);
from_chars_result from_chars(const char* begin, const char* end, unsigned long long& value, int base = 10);
```

Requires: `base` has a value between 2 and 36 (inclusive).

Effects: Digits in the range 10..36 are represented as lowercase characters a..z. If `value` has unsigned integral type, the pattern is a non-empty sequence of digit characters according to `base`. If `value` has signed integral type, the pattern is an optional minus sign followed by a non-empty sequence of digit characters.

```
from_chars_result from_chars(const char* begin, const char* end, float& value, bool hex = false);
from_chars_result from_chars(const char* begin, const char* end, double& value, bool hex = false);
from_chars_result from_chars(const char* begin, const char* end, long double& value, bool hex = false);
```

Effects: If `hex` is false, the pattern is an optional minus sign followed by a non-empty sequence of decimal digit characters with at most one decimal point, and an optional trailing exponent introduced by "e" or "E" followed by an optional minus sign followed by a non-empty sequence of decimal digits. Otherwise, the pattern is an optional minus sign followed by a non-empty sequence of *hexadecimal-digit* characters (2.13.2 [lex.icon]) with at most one radix point, and a trailing exponent introduced by "p" or "P" followed by an optional minus sign followed by a non-empty sequence of decimal digits. In either case, the resulting `value` is one of at most two floating-point values closest to the value of the string matching the pattern. The pattern also matches the representations of non-finite values described for output.

Accessor functions

```
char * get<0>(const to_chars_result& r);
```

Returns: `r.ptr`

```
bool get<1>(const to_chars_result&);
```

Returns: `r.overflow`

```
const char * get<0>(const from_chars_result& r);
```

Returns: `r.ptr`

```
error_code get<1>(const from_chars_result& r);
```

Returns: `r.ec`

Open Issues

- discuss naming again to break the tie between to_text and to_chars
- offer lowercase and uppercase output (for base > 10 and scientific)?
- which header?
- for digits > 9, also parse uppercase letters or just lowercase ones?
- for hexfloats, generate / parse a 0x prefix?
- check the names to_char_result and from_char_result; do we want a _t suffix?
- offer fixed-width integer output with leading zeroes? to_chars(begin, end, value, base=16 width=sizeof(value)*2) is a frequent case for pointers and faster with SIMD than removing the leading zeroes

References

- "How to print floating-point numbers accurately" by Guy L. Steele Jr. and Jon L White, Proceedings of the ACM SIGPLAN'90 Conference on Programming Language Design and Implementation; http://www.kurtstephens.com/files/p372-steele.pdf (starting at page 4 of the PDF)
- "Printing Floating-Point Numbers Quickly and Accurately with Integers" by Florian Loitsch, PLDI'10 http://www.cs.tufts.edu/~nr/cs257/archive/florian-loitsch/printf.pdf
- "How to Read Floating Point Numbers Accurately" by William D Clinger, University of Oregon http://www.cesura17.net/~will/professional/research/papers/howtoread.pdf